==========

==========

CHAPTER 25 — SECURITY ARCHITECTURE & MMU HARDENING

==========

==========

This chapter explains **how the MMU becomes a security boundary**, how ARMv8's architectural features enforce isolation, and how real systems harden their memory layouts against attacks.

****

****

25.1 — Why MMU Hardening Matters

****

****

The MMU is the **primary security barrier** in ARMv8 systems.

If MMU configuration is wrong:

- user processes escape into kernel
- VMs escape into hypervisor

- DMA corrupts memory
- devices leak secrets
- code injection becomes possible
- ROP/JOP attacks succeed
- secure world becomes compromised

MMU hardening is mandatory for:

- OS kernels
- hypervisors
- secure enclaves
- automotive ECUs
- medical devices
- industrial controllers

****

****

25.2 — ARMv8 Security Layers

****

****

ARMv8 security is built on:

Layer 1 — Exception Levels

- EL0 → user
- EL1 → kernel
- EL2 → hypervisor
- EL3 → secure monitor

Layer 2 — Memory Translation

- Stage-1 (EL1)
- Stage-2 (EL2)
- Secure translation (EL3)

Layer 3 — Access Control

- AP
- S2AP
- PAN
- PXN/UXN
- WXN

Layer 4 — Memory Attributes

- Device-nGnRnE
- Normal WB
- Non-exec data

Layer 5 — Hardware Security

- TrustZone
- Pointer Authentication (PAC)
- Branch Target Identification (BTI)

****

****

25.3 — W^X (Write-xor-Execute) Enforcement

****

****

W^X means:

- writable memory must not be executable
- executable memory must not be writable

MMU Enforces W^X Using:

- PXN (Privileged eXecute Never)
- UXN (User eXecute Never)
- AP bits
- S2PXN/S2UXN

Rules

- code pages → executable + read-only
- data pages → writable + non-exec
- stacks → writable + non-exec
- heaps → writable + non-exec
- MMIO → non-exec

Why?

Prevents code injection and ROP/JOP chains.

****

****

25.4 — PAN (Privileged Access Never)

****

****

PAN prevents EL0 from accessing EL1 memory.

Without PAN

User space can accidentally or maliciously access kernel memory.

With PAN

All EL1 mappings are inaccessible to EL0 unless explicitly allowed.

Rules

- enable PAN early in kernel boot
- ensure kernel uses TTBR1_EL1
- ensure user uses TTBR0_EL1

****

****

25.5 — PXN/UXN (Execute Never)

****

****

PXN/UXN prevent execution from:

- data pages
- stacks
- heaps
- MMIO
- DMA buffers
- shared memory

Rules

- PXN for kernel data
- UXN for user data
- S2PXN/S2UXN for guest data

Why?

Prevents execution of injected payloads.

****

****

25.6 — SCTLR_ELx.WXN (Write-xor-Execute Global Policy)

****

****

WXN enforces W^X globally:

- writable pages cannot be executable
- even if PXN/UXN are misconfigured

Best Practice

Enable WXN at EL1, EL2, and EL3.

****

****

25.7 — Stage-2 Security Enforcement

****

****

Hypervisor uses Stage-2 to enforce:

- VM isolation
- device isolation
- memory isolation
- execute permissions
- read/write permissions

Hypervisor Always Overrides Guest

Even if guest marks memory executable, hypervisor can:

- force non-exec
- force read-only
- force Device-nGnRnE
- force Non-Shareable

****

****

25.8 — TrustZone (Secure vs Non-Secure World)

****

****

TrustZone provides hardware isolation:

Secure World

- secure RAM
- secure MMIO
- secure code/data
- secure interrupts

Non-Secure World

- normal RAM
- normal MMIO

Rules

- secure memory must never be mapped in non-secure world
- secure MMIO must be Device-nGnRnE
- secure world must use separate page tables

****

****

25.9 — Pointer Authentication (PAC)

****

****

PAC protects:

- return addresses
- function pointers
- vtables
- jump tables

How It Works

PAC adds cryptographic signatures to pointers.

MMU Interaction

PAC requires:

- non-exec data
- W^X enforcement
- correct PXN/UXN configuration

****

****

25.10 — Branch Target Identification (BTI)

****

****

BTI prevents:

- jump-oriented programming
- branch-target injection
- ROP/JOP chains

MMU Interaction

BTI requires:

- correct executable page attributes
- correct alignment
- correct instruction fetch behavior

****

****

25.11 — Secure Memory Attributes

****

****

Normal WB

- secure RAM
- secure code/data

Device-nGnRnE

- secure MMIO
- secure peripherals

Non-Exec

- secure stacks
- secure heaps

****

****

25.12 — Secure Page-Table Design

****

****

Secure page tables must be:

- isolated
- non-shareable
- non-cacheable for MMIO
- cacheable for RAM
- protected by EL3

Rules

- secure page tables must not be writable by EL1/EL2
- secure page tables must not be accessible from non-secure world

****

****

25.13 — DMA Security

****

****

DMA can bypass the MMU → dangerous.

IOMMU Required

IOMMU enforces:

- DMA isolation
- device memory protection
- secure DMA buffers

Rules

- DMA buffers must be Normal WB or NC
- MMIO must be Device-nGnRnE
- secure DMA must use secure IOMMU context

****

****

25.14 — Device Isolation

****

****

Devices must be isolated:

Hypervisor

- traps MMIO
- enforces S2AP
- enforces S2PXN/S2UXN
- enforces S2AttrIndx

Secure World

- isolates secure MMIO
- blocks non-secure access

****

****

25.15 — Memory Partitioning for Security

****

****

Security partitions:

Secure Partition

- secure RAM
- secure MMIO
- secure code/data

Non-Secure Partition

- normal RAM
- normal MMIO

Hypervisor Partition

- Stage-2 tables
- device virtualization

****

****

25.16 — Hardening Against Common Attacks

****

****

Attack 1 — Code Injection

Mitigation:

- W^X
- PXN/UXN
- WXN
- PAC

Attack 2 — ROP/JOP

Mitigation:

- BTI
- PAC
- non-exec stacks

Attack 3 — DMA Attacks

Mitigation:

- IOMMU
- secure DMA buffers
- device isolation

Attack 4 — VM Escape

Mitigation:

- Stage-2 permissions
- Stage-2 attributes
- VMID isolation

Attack 5 — Kernel Escape

Mitigation:

- PAN
- PXN
- TTBR1 isolation

****

****

25.17 — Hardening QNX/RTOS

****

****

QNX requires:

- deterministic isolation
- strict W^X
- strict MMIO rules
- static partitions
- static ASID ranges
- static VMID ranges

Best Practices

- no dynamic remapping
- no dynamic MMIO mapping
- no overcommit
- no fragmentation

****

****

25.18 — Hardening Hypervisors

****

****

Hypervisors must:

- enforce Stage-2 permissions
- enforce Stage-2 attributes
- isolate devices
- isolate DMA
- isolate VMs
- isolate interrupts

Best Practices

- use blocks for guest RAM
- use pages for MMIO
- enforce S2PXN/S2UXN
- enforce S2AP

****

****

25.19 — Full MMU Hardening Example

****

****

Kernel Code

```
AttrIndx = 0 (Normal WB)
AP = RO
PXN = 0
UXN = 1
```

Kernel Data

```
AttrIndx = 0
AP = RW
PXN = 1
UXN = 1
```

User Code

```
AP = RO
UXN = 0
```

User Data

```
AP = RW
UXN = 1
```

MMIO

```
AttrIndx = 2 (Device-nGnRnE)
PXN = 1
UXN = 1
```

Hypervisor Stage-2

```
S2AP = RO
S2PXN = 1
S2UXN = 1
```

****

****

25.20 — Chapter Summary

****

****

This chapter covered:

- W^X enforcement
- PAN
- PXN/UXN
- WXN

- Stage-2 security
- TrustZone
- PAC
- BTI
- secure memory attributes
- secure page-table design
- DMA security
- device isolation
- partitioning
- attack mitigation
- QNX hardening
- hypervisor hardening
- full examples